

Insurance Costs Calculation Documentation

1 Introduction

The `insurance_costs.py` script calculates operational insurance costs and, when enabled, wildfire liability insurance costs for transmission line assets. Operational premiums are based on insurable asset value (conductors, structures, converters) and are paid annually over the project lifetime. Wildfire liability uses a rate-on-line (ROL) formula: $\text{annual premium} = \text{ROL} \times \text{liability limit}$. Both are written to the batch summary (operational via `add_insurance_costs`; wildfire liability via `add_wildfire_liability_costs`, with zeros when disabled).

2 Method Overview

2.1 Purpose

This script calculates insurance costs by:

1. Loading project details and constructing category via `build_category_string`; loading build costs via `build_costs.load_costs` to get conductor/structure/converter costs with contingencies
2. Determining insurable asset value from which components are insured (`insurable_components`; default all)
3. Applying premium rate from `premium_by_construction_type` or default `premium_rate`
4. Calculating annual premium and lifetime nominal cost for operational insurance
5. Optionally calculating wildfire liability premium ($\text{ROL} \times \text{liability limit}$) when `insurance.wildfire_liability` is true
6. Calculating present value of annual premiums (payment start at COD) for both operational and wildfire liability
7. Writing `add_insurance_costs` and `add_wildfire_liability_costs` (zeros when wildfire liability disabled), then `write_batch_summary`

2.2 Calculation Approach

The insurance cost calculation follows this structure:

$$\text{Insurable Value} = \sum_{\text{insured components}} \text{Component Cost with Contingencies} \quad (1)$$

$$\text{Annual Premium} = \text{Insurable Value} \times \text{Premium Rate} \quad (2)$$

$$\text{Lifetime Nominal Cost} = \text{Annual Premium} \times \text{Project Lifetime} \quad (3)$$

$$\text{Present Value} = \sum_{t=\text{COD}}^{\text{COD}+T-1} \frac{\text{Annual Premium}}{(1+r)^t} \quad (4)$$

where T is the project lifetime, r is the real WACC discount rate, and $\text{COD} = \text{delay_years} + \text{construction_years}$ (payment start at Commercial Operation Date).

Wildfire liability (when enabled): Annual premium = `rate_on_line` × `liability_limit`; nominal lifetime cost = annual premium × project_lifetime; PV same as above starting at COD. Returns None if `wildfire_liability.enabled` is false or parameters invalid.

2.3 Inputs and Outputs

Inputs:

- Build costs (conductor, structure, converter with contingencies) via `build_costs.load_costs`
- Insurance configuration from `load_insurance_details` (04.insurance): `insurance.insurable_comp`, `premium_by_construction_type`, `premium_rate`; `insurance.wildfire_liability.enabled`, `liability_limit`, `rate_on_line`
- Project technical details (construction type, project lifetime, delay years, construction years)
- Financing parameters (WACC for present value)

Outputs:

- Operational: insurable value, annual premium, premium rate, nominal lifetime cost, PV total (via `add_insurance_costs`)
- Wildfire liability: when enabled, same metrics plus liability limit and rate-on-line (via `add_wildfire_liability_costs`); when disabled, zeros written

2.4 Integration with Other Scripts

This script integrates with:

- `build_costs.py` - `load_costs` for conductor/structure/converter costs with contingencies

- `smart_loaders.py` - `load_project_technical_details`, `load_physical_details`, `load_contingencies`, `load_financing_details`, `load_insurance_details`, `get_project_data_raw`
- `calculation_utils.py` - `build_category_string`
- `financial_utils.py` - `calculate_present_value`, `calculate_cod_year`
- `constants.py` - `TRANSMISSION_TYPE_DC` for DC check
- `smart_output.py` - `CTCCOutputManager` (`add_insurance_costs`, `add_wildfire_liability_costs`, `write_batch_summary`)

3 Key Functions

3.1 `calculate_insurance_costs(insurance_yaml, conductor_cost_with_contingencies, structure_cost_with_contingencies, converter_cost_with_contingencies, construction_type, project_lifetime)`

Calculates operational insurance costs based on insurable asset value.

Parameters:

- `insurance_yaml` (dict): Loaded insurance YAML data
- `conductor_cost_with_contingencies` (float): Conductor cost with contingencies
- `structure_cost_with_contingencies` (float): Structure cost with contingencies
- `converter_cost_with_contingencies` (float): Converter cost with contingencies
- `construction_type` (str): Type of construction (overhead, underground, subsea)
- `project_lifetime` (int): Project lifetime in years

Returns:

- Dictionary containing:
 - `insurable_value` (float): Total insurable asset value
 - `annual_premium` (float): Annual insurance premium
 - `premium_rate` (float): Premium rate applied
 - `nominal_lifetime_cost` (float): Total lifetime cost in nominal terms

Key Logic:

```

1 # Calculate insurable value based on which components are insured
2 insurable_value = 0.0
3 if components.get("conductors", True):
4     insurable_value += conductor_cost_with_contingencies
5 if components.get("structures", True):
6     insurable_value += structure_cost_with_contingencies
7 if components.get("converters", True):
8     insurable_value += converter_cost_with_contingencies
9
10 # Get premium rate (construction type specific or default)
11 premium_rate = premium_by_type.get(
12     construction_type.lower(),
13     ins_cfg.get("premium_rate", 0.002)
14 )
15
16 # Calculate annual premium
17 annual_premium = insurable_value * premium_rate
18 nominal_lifetime_cost = annual_premium * project_lifetime

```

3.2 calculate_wildfire_liability_premium(insurance_yaml, project_lifetime)

Calculates wildfire liability insurance premium using rate-on-line (ROL). Returns None if insurance.wildfire_liability.enabled is false or liability_limit / rate_on_line ≤ 0.

Returns: Dict with liability_limit, rate_on_line, annual_premium, nominal_lifetime_cost, or None.

Key logic:

```

1 if not wildfire_liability.get("enabled", False):
2     return None
3 liability_limit = wildfire_liability.get("liability_limit", 0)
4 rate_on_line = wildfire_liability.get("rate_on_line", 0)
5 if liability_limit <= 0 or rate_on_line <= 0:
6     return None
7 annual_premium = rate_on_line * liability_limit
8 nominal_lifetime_cost = annual_premium * project_lifetime
9 return {"liability_limit": ..., "rate_on_line": ..., "annual_premium": ..., "nominal_lifetime_cost": ...}

```

3.3 main()

Main execution function that orchestrates the insurance cost calculation and output.

Workflow:

1. Loads project technical details; constructs category via build_category_string; gets number_of_converters from get_project_data_raw if DC

2. Loads build costs via `load_costs(category, total_miles, number_of_converters, contingencies, reconductoring)`; loads insurance via `load_insurance_details()`
3. Calls `calculate_insurance_costs(...)` and `calculate_wildfire_liability_premium(insurance_costs, project_lifetime)`
4. `insurance_start_year = calculate_cod_year(delay_years, construction_years)`;
`insurance_pv = calculate_present_value(annual_premium, wacc_real, project_lifetime, insurance_start_year)`; same for wildfire liability PV if enabled
5. `add_insurance_costs(csv_results)`; `add_wildfire_liability_costs(...)` (with zeros when disabled); `write_batch_summary()`

4 Example

The following example demonstrates a typical usage of the insurance cost calculation:

```

1 # Example: 500 MW Overhead AC line
2 # Category: "Overhead/AC/500MW/Advanced Aluminum/NA"
3 #
4 # Inputs:
5 #   - Conductor cost (with contingencies): $55M
6 #   - Structure cost (with contingencies): $25M
7 #   - Converter cost: $0 (AC project)
8 #   - Premium rate: 0.2% (overhead construction)
9 #   - Project lifetime: 50 years
10 #   - Payment start: Year 3 (after 2-year construction)
11 #
12 # Calculation:
13 #   Insurable value = $55M + $25M + $0 = $80M
14 #   Annual premium = $80M * 0.002 = $160,000
15 #   Lifetime nominal = $160,000 * 50 = $8M
16 #   Present value = $160,000 * PV annuity (50 years, 3% discount) =
17 #   $4.2M
18 #
19 # Output:
20 #   - Insurable value: $80M
21 #   - Annual premium: $160,000
22 #   - Lifetime nominal: $8M
23 #   - Present value: $4.2M

```

Integration Example:

```

1 # When called from ctcc.py:
2 # 1. Script loads build costs from build_costs.py
3 # 2. Calculates insurance costs
4 # 3. Writes to CSV via CTCCOutputManager:
5 csv_manager = CTCCOutputManager()

```

```

6 csv_results = {
7     "annual_premium": results["annual_premium"],
8     "nominal_lifetime_cost": results["nominal_lifetime_cost"],
9     "pv_total": insurance_pv,
10    "insurable_value": results["insurable_value"],
11    "premium_rate": results["premium_rate"],
12 }
13 csv_manager.add_insurance_costs(csv_results)
14 if wildfire_liability_results:
15     csv_manager.add_wildfire_liability_costs(wildfire_csv_results)
16 else:
17     csv_manager.add_wildfire_liability_costs({"annual_premium": 0, "
18         nominal_lifetime_cost": 0, "pv_total": 0, "liability_limit":
19         0, "rate_on_line": 0})
20 csv_manager.write_batch_summary()

```

5 Key Implementation Details

5.1 Insurable Components

The script allows configuration of which components are insurable (conductors, structures, converters). By default, all components are insured, but this can be customized in the YAML configuration.

5.2 Premium Rate by Construction Type

Different construction types may have different premium rates. The script first checks for a construction-type-specific rate, then falls back to a default rate if not specified.

5.3 Payment Timing

Insurance premiums start at Commercial Operation Date (COD), which is after the delay and construction periods. Premiums are paid annually for the project lifetime.

5.4 Wildfire Liability

Wildfire liability insurance is optional and controlled by `insurance.wildfire_liability.enabled` in the insurance YAML. When enabled, $\text{annual premium} = \text{rate_on_line} \times \text{liability_limit}$. The script always writes `add_wildfire_liability_costs` (with zeros when disabled) so batch summary columns remain consistent.

5.5 AFUDC Treatment

Operational insurance is not AFUDC-eligible because it is an operating expense, not a construction cost. Only construction-period costs are eligible for AFUDC capitalization.

5.6 Dependency on Build Costs

This script requires build costs to be calculated first (via `build_costs.py`) to determine insurable asset values. It imports and calls `load_costs()` from `build_costs.py`.